FA2 DEVOPS PROJECT GUIDE

CampusDesk

Smart Campus Maintenance and Service Request System, explained in plain language.

In one sentence. Students report broken fans, dead projectors, and bad Wi-Fi. Administrators track those reports until they are fixed. The same repository also shows how that software is built, tested, packed, deployed, and watched.

Developer · GitHub · GitHub Actions · Build · Test · Docker · Terraform · Kubernetes · App · Prometheus · Grafana

College demonstration project
Two apps, one delivery pipeline

Marks focus: FA2 rubric, 10 marks
Excellent column explained inside

START HERE

What this project actually is

CampusDesk is a small college website for campus repairs, wrapped in a full DevOps delivery path. The website is the product. The pipeline is what the FA2 rubric is mainly grading.

The product

A student types a problem: title, description, category, location, and priority. An administrator later changes the status to In Progress, Resolved, or Rejected, and can write how it was fixed.

The DevOps story

Code lives on GitHub. A workflow builds it, tests it, puts it in Docker images, uses Terraform to place it on Kubernetes, and checks that Prometheus can see the running API.

> There is no separate login server and no machine-learning part. The header switch between Student and Admin is a demonstration role, enough for a viva. A real college system would put proper login in front of the same API.

What you can open

Screen	Address in the app	Who it is for
Introduction	/	Anyone
Student dashboard	/dashboard	Student
New request	/requests/new	Student
My requests	/requests	Student
One request	/requests/:id	Student or admin
Admin dashboard	/admin	Admin
All requests	/admin/requests	Admin

Contents of this guide: the idea, every important word, what was built, the live numbers on the Grafana board, and the Excellent column of the 10-mark FA2 rubric.

THE IDEA

The problem, and the queue that replaces chat

On a campus, small failures are constant: a socket sparks, a lab PC will not boot, a projector stays dark, a chair is cracked, a washroom needs cleaning. If those reports live in WhatsApp groups, they get buried. Nobody can see what is still open, what is urgent, or who already reported the same thing.

CampusDesk keeps one queue. Every report has a place, a category, a priority, and a status. Facilities staff do not hunt through messages. They open the admin table, filter, and update the row.

One request, from typing to “resolved”

Student form → **API checks the fields** → **Row saved as Pending** → **Admin changes status** → **Notes stored**

- **Categories:** Electrical, Wi-Fi, Computer/Lab, Classroom, Projector, Furniture, Cleanliness, Other.
- **Priorities:** Low, Medium, High, Critical.
- **Statuses:** Pending, In Progress, Resolved, Rejected. A new request always starts as Pending.

The student name in the header is stored in the browser and sent with each new request, so “My Requests” can show that person’s reports. Sample names in the seed data are Aisha Khan, Rahul Menon, and Neha Iyer.

HOW THE PIECES TALK

Architecture, without the jargon first

Think of three rooms. The browser is the front desk. The API is the office that checks forms and writes in the register. The database is the register. Two extra people stand outside and watch the office: Prometheus writes down how busy the office is, and Grafana turns those notes into charts.

Browser → **Frontend** → **Backend API** → **Postgres**

Prometheus **asks** **GET /metrics** **then** **Grafana** **reads** **Prometheus**

Piece	Job	Talks to
Frontend	Student and admin screens. In Docker and Kubernetes, Nginx serves the pages.	Backend, through /api
Backend	REST API, checks input, writes logs, exposes /metrics.	Postgres
Database	Stores the request rows. Locally this is Supabase. The CI cluster can start its own Postgres.	Backend only
Prometheus	Collects numbers from the API every 15 seconds.	Backend
Grafana	Dashboard: rate, errors, latency, uptime, CPU, memory.	Prometheus

> The browser never sees the database password. In containers, Nginx inside the frontend forwards /api and /metrics to the service named backend. The connection string stays on the API.

WORDS YOU WILL BE ASKED

Every term, in easy language

Read these in order. Later sections use the same words.

Frontend THE SCREENS

The part people click. Here it is a React app written in TypeScript, styled with Tailwind, built by Vite. React draws the pages. React Router changes the address without reloading the whole site. Axios sends HTTP calls to the API.

Backend THE OFFICE

A Node.js program using Express. It receives requests such as “create this report” or “list pending reports”, checks them, and talks to the database. TypeScript is JavaScript with declared types, so many mistakes show up before the program runs.

REST API A MENU OF URLS

An agreed set of addresses and methods. GET reads. POST creates. PUT replaces fields. PATCH changes status. DELETE removes a row. A successful answer uses success, message, and data. A bad answer uses success: false plus a message.

Database and Postgres THE REGISTER

Postgres is a relational database: tables, rows, and columns. The requests table holds title, description, category, location, priority, status, resolution notes, the student name, and timestamps. The API creates that table when it starts.

Supabase HOSTED POSTGRES

A hosted Postgres service. For a laptop demo, the API uses a connection string called DATABASE_URL. The password stays in .env or in a Kubernetes Secret. It is not written into the source code. SSL means the connection is encrypted on the way to Supabase.

Environment variable A SETTING OUTSIDE THE CODE

PORT, DATABASE_URL, FRONTEND_URL, and VITE_API_URL are settings. Change the machine, change the setting, leave the code alone. .env.example shows the names. The real .env is not committed.

Validation, Zod, Helmet, CORS THE DOOR CHECKS

Zod rejects a title that is too short or a category that is not on the list. Helmet sets safer HTTP headers. CORS decides which websites may call the API. One error handler turns failures into the same JSON shape. Logs record time, method, route, status, and duration. They do not record passwords or the database URL.

Git and GitHub HISTORY AND THE SHARED COPY

Git remembers every saved change. A commit is one saved snapshot with a message. A branch is a side line of work. GitHub stores that history online. A pull request is a proposal: “please merge this branch.” The rubric’s source stage starts here.

CI/CD ROBOTS THAT REPEAT THE BORING STEPS

Continuous integration means every push is built and tested automatically. Continuous delivery or deployment means the same robot can also ship the result. Doing it by hand once is a demo. Doing it in a workflow is a pipeline.

GitHub Actions THE ROBOT WE CHOSE

A workflow file, .github/workflows/ci-cd.yml, lists jobs and steps. It runs on GitHub’s machines when code is pushed to main or master, and on pull requests. Jenkins is another tool that can do this job. The rubric accepts either. This project uses GitHub Actions.

Build TURN SOURCE INTO SOMETHING RUNNABLE

npm run build compiles the TypeScript frontend into static files and the backend into JavaScript in dist/. Docker build then packs those results into images.

Test and lint PROVE IT, THEN CHECK THE TYPES

Backend tests use Jest and Supertest against an in-memory Postgres (pg-mem), so they do not need Supabase. They cover health, metrics, create, read, validation, and an admin status update. Frontend tests use Vitest and React Testing Library for the student dashboard, the form, and the request card. Lint here runs the TypeScript compiler in strict mode.

Docker, image, container A PACKED LUNCH BOX

An image is a packed snapshot of the app plus what it needs to run. A container is that image actually running. The backend image compiles TypeScript, then runs node dist/server.js as a normal user, not as root. The frontend image builds the site, then Nginx serves the files. Multi-stage means the compiler tools are left behind in the final image.

Docker Compose

START SEVERAL BOXES TOGETHER

docker compose up --build starts the API, the website, Prometheus, and Grafana on one machine. The database stays on Supabase unless you point the API elsewhere. Useful local addresses: website :3000, API health :5000, metrics :5000/metrics, Prometheus :9090, Grafana :3001.

Nginx and reverse proxy

THE RECEPTIONIST

Nginx serves the React files. When the browser asks for /api or /metrics, Nginx forwards that call to the backend container. The browser stays on one address. That is a reverse proxy.

Kubernetes

A SCHEDULER FOR CONTAINERS

Kubernetes runs containers, restarts them if they die, and gives them stable names. A manifest is a YAML file that says what you want: “two concepts matter — the desired state, and the cluster making reality match it.”

Pod, Deployment, Service

THE RUNNING UNIT, THE MANAGER, THE ADDRESS

A pod is one or more containers that run together. A Deployment keeps the right number of pods alive and rolls out a new image. A Service is a stable name and port in front of changing pods. NodePort publishes that service on a fixed port of the cluster node, so a viva can open the site from the laptop.

Namespace, ConfigMap, Secret

A FOLDER, SETTINGS, AND PASSWORDS

The namespace smart-campus keeps this app’s objects together. A ConfigMap holds non-secret settings such as the port and allowed website origin. A Secret holds DATABASE_URL. Terraform creates that Secret. The password file is not applied from git.

Liveness and readiness

IS IT ALIVE, AND IS IT READY

A liveness probe asks “is the process still healthy?” Kubernetes restarts it if not. A readiness probe asks “should I send users yet?” The API exposes /api/health/live for liveness and a fuller /api/health that also reports the database. Deployments also set CPU and memory requests and limits so one container cannot silently eat the machine.

Minikube

KUBERNETES ON YOUR LAPTOP

A real cluster is many machines. Minikube is one small cluster inside Docker on a single computer, which is enough to show pods, services, and a deploy. GitHub Actions starts Minikube for the same reason: without a cluster, deployment would only be a file check.

Terraform INFRASTRUCTURE AS CODE

Terraform reads .tf files and creates the namespace, workloads, and database secret through the Kubernetes provider. You describe the end state. Terraform applies it. Ansible is another allowed tool for this slot. This project uses Terraform, not Ansible. use_in_cluster_database=true also starts Postgres inside the cluster, which is what the CI job does so the pipeline does not depend on a personal Supabase password.

Monitoring, Prometheus, scrape ASKING "HOW ARE YOU?" ON A TIMER

Monitoring is watching a running system with numbers, not by staring at a terminal. Prometheus pulls those numbers. A scrape is one pull. Every 15 seconds it calls GET /metrics on the backend. A target is UP when that pull succeeds.

Metric, counter, gauge, histogram THREE KINDS OF NUMBER

A counter only goes up, such as total HTTP requests. A gauge moves up and down, such as requests in progress right now. A histogram sorts durations into buckets so you can ask "how slow were the slowest 5 percent?" Labels such as method, route, and status code let one metric split into several series. Routes use the pattern /api/requests/:id, not the real id, so one student does not create a brand-new series.

Grafana THE CHARTS

Grafana does not collect the numbers itself. It queries Prometheus and draws the Smart Campus Maintenance dashboard. The datasource is provisioned automatically in both Compose and Kubernetes, so you do not click through a setup wizard during the demo.

Health check A SIMPLE "I AM HERE"

GET /api/health returns status, service name, time, uptime, and database state. It is the first curl in a viva. /metrics is the longer page Prometheus reads. Scrapes of /metrics are not counted as normal user traffic, so the dashboard is not polluted by the monitor watching itself.

WHAT WAS BUILT

What we did, from the app to the pipeline

The work is one repository, smart-campus-devops, with two applications and a delivery folder. Nothing here is a pile of unused microservices.

Folder	What it is
frontend/	React screens for students and administrators.
backend/	Express API, database access, logging, and Prometheus metrics.
devops/kubernetes/	Manifests for the website, API, Prometheus, and Grafana on Minikube.
devops/terraform/	The deploy step. Applies those manifests and can create in-cluster Postgres.
devops/monitoring/	Prometheus scrape config and the Grafana dashboard JSON.
devops/scripts/	Waits until the Prometheus target smart-campus-backend is UP.
.github/workflows/	The CI/CD pipeline.
docs/	Architecture note, viva script, this guide.

1. The application

Students can create a request and see their own list. Administrators see counts (total, pending, in progress, resolved, rejected, critical), then search, filter, update, or delete. The API validates every write. Tests cover the important paths without a real cloud database.

2. Packing it

Each app has its own Dockerfile. Compose runs the website, API, Prometheus, and Grafana together. Locally the API still uses Supabase, so you do not install Postgres on the laptop for the Compose demo.

3. Describing the cluster

Kubernetes YAML describes the namespace, deployments, services, ConfigMap, probes, and resource limits. kubeconform checks those files against Kubernetes schemas even when no cluster is running. That check is necessary, and it is not the deploy itself.

4. Deploying with Terraform

terraform apply creates the objects. For a Supabase demo you pass database_url. For the automated pipeline you set use_in_cluster_database=true, so Postgres starts inside smart-campus

and the API points at it. Images are loaded into Minikube first, and imagePullPolicy: IfNotPresent uses those local images.

5. The pipeline that does this every time

On a push or pull request, GitHub Actions runs this sequence:

1. Checkout the source.
2. Install frontend and backend dependencies with npm ci.
3. Lint and test the frontend. Test the backend.
4. Build both apps.
5. Build both Docker images and check that the backend module loads and the frontend answers HTTP.
6. Check Compose config, kubeconform, and terraform validate.
7. On a push, push images to Docker Hub when those secrets exist. If the secrets are missing, the step says so and the workflow still continues.
8. Start Minikube, load the images built in that job, and terraform apply with Postgres inside the cluster.
9. Wait until Prometheus reports the backend target as UP.

> That last step is the difference between “we wrote a dashboard” and “the pipeline proved monitoring is attached.” The script port-forwards Prometheus and reads its targets API until job smart-campus-backend has health up.

THE NUMBERS ON THE DASHBOARD

Metrics, explained one by one

These are the application metrics, the numbers Prometheus stores. They are separate from the mark scheme in the next section. The API exposes them at GET /metrics using the prom-client library.

Name	Kind	What it means in plain words
http_requests_total	Counter	How many HTTP calls have finished, split by method, route, and status code. 200 is success. 400 is a bad form. 500 is the server failing.
http_request_duration_seconds	Histogram	How long those calls took. The dashboard uses this for p95: 95 percent of recent calls were faster than this number. The remaining 5 percent were slower.
http_active_requests	Gauge	How many calls are inside the API at this moment. It goes up when a call starts and down when it finishes.
app_uptime_seconds	Gauge	How long this API process has been running. A sudden drop to a small number means the process restarted.
process_cpu_user_seconds_total	Counter	CPU time used by the Node.js process. The chart shows the rate, which is "how busy is the CPU right now."
process_resident_memory_bytes	Gauge	Memory the process is holding. A line that only climbs can mean a leak. A steady line is what you hope to see in a short demo.

What each Grafana panel is asking

- **HTTP requests** — the total so far: sum(http_requests_total).
- **Request rate** — calls per second over the last minute. rate() turns a counter into a speed.
- **Error rate** — the same speed, but only status codes 500–599. A flat zero during the demo is a healthy API.
- **Response time p95** — histogram_quantile(0.95, …). This is the "how slow is slow" number, in seconds.
- **Request rate by status** — the same speed, drawn as separate lines for 200, 400, and so on.
- **CPU and memory** — whether the process itself is struggling, not whether a student clicked the right button.

The board refreshes every 5 seconds and looks at the last 15 minutes. Click around the website during the viva so the request-rate panel moves. Prometheus targets live at port 9090, path /targets. You want smart-campus-backend to say UP.

THE MARK SCHEME FROM THE RUBRIC

The Excellent column, and how this project meets it

The sheet shared for this subject is **FA2 Rubrics (10 Marks)**. Three rows. The word Excellent is the top performance level in each row. This section is that matrix, then a direct reading of this repository against the Excellent cell.

Row 1 — Selection of methods, tools, and design

Allocated **3 marks**. Linked outcomes on the sheet: PO1, PO2, PO3, PO12. In the usual engineering-outcome wording, those are engineering knowledge, problem analysis, design of a solution, and the ability to keep learning. Your college's official PO sentences are the ones printed in the syllabus.

Level	What the sheet asks	Marks
Excellent	All pipeline stages (source, build, test, containerization, deployment, monitoring) are designed for the requirements, and modern tools are selected: Jenkins or GitHub Actions, Docker, Kubernetes, Ansible or Terraform, Prometheus or Grafana.	3
Very Good	All stages are designed for the requirements, but the design still needs improvement, with modern tools selected.	2.5
Fair	Stages are designed only moderately, and modern DevOps tools are not selected.	1.5–2
Poor	Some stages are missing, and modern tools are not selected.	0.5–1

> **Where CampusDesk sits against Excellent.** The stages are designed in this order: GitHub source, npm and Docker build, automated tests, Docker images, Terraform onto Kubernetes, Prometheus scrape plus Grafana. The tool pairs are filled by one choice from each pair: GitHub Actions, Docker, Kubernetes on Minikube, Terraform, and both Prometheus and Grafana. Ansible and Jenkins are the other legal choices. You do not need both members of a pair.

Row 2 — Implementation

Allocated **5 marks**. This is the heavy row. Outcomes on the sheet: PO1, PO2, PO3, PO4, PO5, PO12. PO4 is investigation (tests and the Prometheus check are the evidence). PO5 is modern tool usage.

Level	What the sheet asks	Marks
Excellent	A working end-to-end CI/CD pipeline: automated build, test, deployment, and monitoring, using modern tools.	5
Very Good	Meets the requirements but still needs improvement, with modern tools selected.	4
Fair	Only moderate accordance, and modern tools are not selected.	2–3
Poor	Not in accordance with the requirements, and modern tools are not selected.	1

Excellent on this row is not “the YAML exists.” It is a pipeline that actually does the four verbs. In this repository those verbs are steps in ci-cd.yml:

Verb on the rubric	What the workflow does
Build	npm run build for both apps, then docker build for both images.
Test	Frontend lint and test, backend test, then a live check that each image starts.
Deployment	Minikube starts, images are loaded, terraform apply creates the namespace and workloads, including Postgres for that run.
Monitoring	Prometheus and Grafana are part of that apply. The next step blocks until the backend target is UP.

A green Actions run that includes the line “Prometheus target is UP” is the evidence for this cell. File checks alone (kubeconform, terraform validate) support the design row. They do not replace the deploy.

Row 3 — Timely submission

Allocated **2 marks**. Outcomes on the sheet: PO8 and PO12. PO8 is ethics: secrets stay out of git, logs do not print passwords, and the demo role switch is described honestly as a demo.

Level	What the sheet asks	Marks
Excellent	Submitted before the deadline, with an effective solution and a demonstration, plus appropriate contents, documentation, and modules.	2
Very Good	On time, effective solution and demonstration, but some improvement still needed.	1.5
Fair	Completed and submitted, with a fair solution and demonstration that still need improvement.	1
Poor	Late submission, satisfactory solution, unsatisfactory demonstration.	0.5

The modules are the frontend, the backend, Kubernetes, Terraform, monitoring, and the workflow. The documentation is the README, the architecture note, and the viva script in docs/presentation/demo-script.md. The 2 marks also depend on submitting before the deadline and walking through a live demo. The repository cannot earn that clock for you.

Reading the total

Row	Excellent mark	What earns it here
1. Design and tools	3	Six stages designed, and one tool from each required pair.
2. Working pipeline	5	Actions builds, tests, deploys with Terraform to Minikube, and proves the Prometheus target is UP.
3. Submission and demo	2	On-time hand-in, docs, modules, and a short live walkthrough.
Total	10	3 + 5 + 2. The examiner awards the marks. This table is the map, not a grade already given.

IF SOMEONE ASKS “SO WHAT?”

Short answers you can say out loud

What is the project?

A campus maintenance desk. Students file repair requests. Admins update status. Underneath, a CI/CD pipeline builds, tests, containerizes, deploys, and monitors that app.

Why DevOps, if the website already works?

Because the rubric grades the path, not only the form. A working form with no automated deploy and no monitoring is a lower row. The pipeline is the submission.

Why Minikube if GitHub Actions already runs it?

Actions proves the pipeline on GitHub’s machine. Minikube on your laptop is how you show pods and the site in the room. Same manifests. Two places to run them.

Why Terraform if the YAML already exists?

The YAML is the Kubernetes description. Terraform is the tool the rubric asks for in the Ansible-or-Terraform slot, and it is the command the pipeline runs to create that description on the cluster.

What should the demo order be?

Git history, tests, docker compose and a status change in the UI, curl health and metrics, Prometheus targets, Grafana, the Actions log, then Minikube pods. Close by repeating the path: GitHub, Actions, build, test, Docker, Terraform, Kubernetes, application, Prometheus, Grafana.

CampusDesk guide for the Smart Campus Maintenance project. Terms match the repository: React and Express applications, Supabase or in-cluster Postgres, Docker, Terraform, Kubernetes on Minikube, Prometheus, Grafana, and GitHub Actions. The mark table follows the FA2 rubric sheet (10 marks) with the Excellent, Very Good, Fair, and Poor columns.